

NESTJS ≠ NEXT.JS

NestJS: server-side Node framework (Angular-style modules + DI, on Express/Fastify) for APIs, GraphQL, WebSockets, microservices. **Next.js:** Vercel's React framework (SSR/SSG/RSC). Common combo: Next.js front end/BFF → NestJS API, shared types or OpenAPI/GraphQL client.

CLI

```
npm i -g @nestjs/cli
nest new app # ESM+Vitest or CJS+Jest
nest g module|controller|service books
nest g resource books # alias: g res
nest g app|lib name # monorepo mode
nest start --watch # npm run start:dev
nest build [--builder swc|rspack|tsc]
// nest build --parallel # monorepo
nest add <pkg> nest info nest deploy (Mau)
nest upgrade [--dry-run]
Aliases: mo co s pi gu itc f d cl r
nest-cli.json
compilerOptions, projects, @app/* paths
oxlint default,
bun supported
```

BOOTSTRAP

```
const app = await NestFactory.create(AppModule);
// Fastify: createNestFastifyApplication(<
// AppModule, new FastifyAdapter()>)
app.setGlobalPrefix('api', { exclude: ['health']
});
app.enableVersioning({ type: VersioningType.URI });
app.enableShutdownHooks();
app.enableCors({ origin: [...] });
app.useGlobalPipes(new ValidationPipe(...));
await app.listen(3000);
Security headers (helmet or built-in), CORS, CSRF: see
Security. Standalone: createApplicationContext; hybrid:
connectMicroservice + startAllMicroservices.
```

BUILDING BLOCKS

```
@Module({ imports, controllers, providers, exports
})
@Global() // use sparingly
@Controller('books') @Get('/:id') @Post()
@Put @Patch @Delete @All @Search QUERY (11.2)
@param('id') @Query() @Body() @Headers() @Req()
@Res({ passthrough: true })
@HttpCode(204) @Header(k, v) @Redirect(url)
@Injectables() constructor(private s: BooksService)
{}
Express 5 paths: '*splat' '{*splat}' '/a/{:id}'
Custom providers
{ provide: T, useClass | useValue | useFactory,
inject: [Dep], useExisting: Other }
@Inject('TOKEN') @Optional() scope: Scope.REQUEST
```

REQUEST LIFECYCLE

middleware **guards** **interceptors ↓** **pipes** **handler**
interceptors ↑ **filters**
Middleware: global → module. Guards/interceptors/pipes:
global → controller → route. Filters run in reverse (route →
controller → global) on exceptions. Bind with @UseGuards
@UseInterceptors @UsePipes @UseFilters or APP_GUARD
APP_INTERCEPTOR APP_PIPE APP_FILTER (DI-aware);
app.useGlobal* has no DI.

PIPES & VALIDATION

Built-in: ParseIntPipe ParseFloatPipe ParseBoolPipe
ParseArrayPipe ParseUIDPipe ParseEnumPipe
ParseDatePipe DefaultValuePipe ParseFilePipe
ValidationPipe
new ValidationPipe({ whitelist: true,
forbidNonWhitelisted: true, transform: true,
transformOptions: { enableImplicitConversion:
true },
errorFormat: 'list'|'grouped',
disableErrorMessage: })
@Body({ schema }) StandardSchemaValidationPipe //
Zod/Valibot/ArkType
class-validator + class-transformer (@Type, mapped types
PartialType PickType OmitType IntersectionType) vs
Standard Schema (no decorators, one schema = type +
validation).

EXCEPTIONS

```
throw new NotFoundException('msg', { cause,
description });
new HttpException(body, 409, { cause }); // v12
errorCode
@Catch(DomainError) class F implements
ExceptionHandler {
catch(e, host: ArgumentsHost) {...} }
@Catch() + HttpAdapterHost // catch-all, adapter-
agnostic
extends BaseExceptionHandler IntrinsicException
(not logged)
Built-ins: BadRequest 400 · Unauthorized 401 ·
Forbidden 403 · NotFound 404 · Conflict 409 · Gone
410 · UnprocessableEntity 422 · TooManyRequests 429
· InternalServerError 500. Map domain errors to HTTP
codes; consider RFC 9457 problem details.
```

FUNDAMENTALS

```
class M extends ConfigurableModuleBuilder<0>()
.setClassName('forRoot').build() {}
static forRoot(o): DynamicModule { module,
providers, exports }
@Injectables({ scope: Scope.REQUEST | TRANSIENT,
durable: true })
forwardRef(() => X) @Inject(forwardRef(...))
ModuleRef.get | resolve | create ; LazyModuleLoader
Reflector.createDecorator<T>()
DiscoveryService.createDecorator
ExecutionContext: getHandler getClass switchToHttp
getType
Hooks (in order): onModuleInit →
onApplicationBootstrap → onModuleDestroy →
beforeApplicationShutdown → onApplicationShutdown
(needs enableShutdownHooks()). Request scope bubbles up
the injection chain: costs performance.
```

APPLICATION

```
ConfigModule.forRoot({ isGlobal: true,
validationSchema })
config.getOrThrow<string>('JWT_SECRET') // never
hardcode
ClassSerializerInterceptor @Exclude() @Expose()
@SerializeOptions
Logger ConsoleLogger bufferLogs
app.useLogger(pino)
EventEmitter2 @OnEvent('x') @Cron() @Interval()
@Timeout()
BullModule.registerQueue() @Processor() extends
WorkerHost
HttpModule firstValueFrom(http.get(url))
```

DATA

```
TypeOrmModule.forRootAsync(...) forFeature([Book])
@InjectRepository(Book) DataSource migrations
(CLI)
PrismaClient in a service onModuleInit $connect
(pin prisma@7)
Drizzle: @nestjs/drizzle (pre-1.0) MikroORM
Sequelize
mongooseModule.forRoot(uri) @Schema() @Prop()
@InjectModel()
CacheModule.register({ ttl: 5000 }) // ms (cache-
manager 7)
@UseInterceptors(CacheInterceptor) @CacheKey
@CacheTTL
```

HTTP

```
cookies: built-in (12.1) or cookie-parser; sessions
FileInterceptor('file') + ParseFilePipe //
Fastify: MultipartModule
return new StreamableFile(stream)
@sse('events') → Observable<MessageEvent>
@Render('view') ServeStaticModule compression
@nestjs/webhooks (pre-1.0) raw body: rawBody: true
FastifyAdapter @fastify/middie Express vs Fastify
hooks
```

SECURITY

```
JwtModule.registerAsync + ConfigService // openssl
rand -hex 32
@nestjs/passport strategies @nestjs/authentication
(pre-1.0)
@Public() + global APP_GUARD @Roles() CASL
policies
ThrottlerModule.forRoot([{ ttl: 60000, limit: 10
}]) @SkipThrottle
argon2 / bcrypt hashing crypto.scripts helmet
CSRF
Never hardcode secrets; validate input; least privilege;
httpOnly + Secure + SameSite cookies.
```

OPENAPI

```
SwaggerModule.createDocument(app, new
DocumentBuilder()
.setTitle().addBearerAuth().build())
SwaggerModule.setup('docs', app, doc)
@ApiTags @ApiOperation @ApiResponse @ApiProperty
@ApiBearerAuth @ApiBody @ApiQuery @ApiParam
@ApiExtraModels
CLI plugin (nest-cli.json) infers properties;
Standard Schema
```

GRAPHQL

```
GraphQLModule.forRoot<ApolloDriverConfig>({ driver:
ApolloDriver,
autoSchemaFile: true }) // code-first; GraphQL
default
@ObjectType @Field @InputType @ArgsType @Resolver
@Query @Mutation @Subscription @ResolveField @Args
subscriptions: graphql-ws (subscriptions-transport-
ws removed)
federation: ApolloFederationDriver
@Directive('@key(...)')
complexity, depth limits, DataLoader for N+1
```

WEBSOCKETS

```
@WebSocketGateway({ cors }) @WebSocketServer()
server
@SubscribeMessage('chat') @MessageBody()
@ConnectedSocket()
OnGatewayInit | Connection | Disconnect
(handleDisconnect reason)
Socket.IO (default) or ws adapter
guards/pipes/filters apply
request-scoped gateways
```

MICROSERVICES

```
NestFactory.createMicroservice(M, { transport:
Transport.RMQ,
options: { urls, queue } })
@MessagePattern('p') req/res @EventPattern('e')
fire & forget
client.send(p, d) (cold Observable)
client.emit(e, d)
ClientsModule.register / ClientProxy hybrid app
TCP Redis NATS (v3: @nats-io/transport- MQTT
node)
RMQ Kafka gRPC (GrpcExceptionHandler) custom
```

CQRS & RELIABILITY

```
CommandBus / QueryBus / EventBus @CommandHandler
@QueryHandler
@EventsHandler AggregateRoot Saga (@nestjs/cqrs
12)
@nestjs/resilience idempotency outbox workflows
locks
(all pre-1.0: resilience, idempotency, outbox,
workflows, locks)
```

TESTING

```
const m = await Test.createTestingModule({ imports
})
.overrideProvider(S).useValue(fake)
.useMocker(createMock).compile();
app = m.createNestApplication(); await app.init();
await
request(app.getHttpServer()).get('/books').expect(200);
m.get(S) m.resolve(S) (scoped) Vitest (ESM) /
Jest (CJS)
GraphQL / WS / microservice: test via client or
supertest
```

PRODUCTION

```
TerminusModule HealthCheckService @HealthCheck()
HealthIndicatorService (legacy HealthIndicator
removed)
@nestjs/observe (pre-1.0) OpenTelemetry prom-
client
FROM node:24-alpine AS build / runtime: npm ci --
omit=dev
node dist/main.js non-root user graceful shutdown
serverless: @codegenie/serverless-express nest
deploy (Mau)
Fastify for throughput; compression at the proxy;
PM2/NGINX
```

CHANGED SINCE OLDER TUTORIALS

v11 **v11.2** **v12.0** **v12.1**
v11: Express 5, path-to-regexp v8 (*splat), Keyv-based
cache with TTL in ms. **v11.2:** HTTP QUERY, SSE signal. **v12.0**
(2026-08): ESM-first packages, nest new ESM+Vitest, oxlint,
Rspack, Standard Schema, HttpException errorCode,
NATS v3, Terminus indicator API, subscriptions-
transport-ws removed, Joi 18. **v12.1** (2026-09): built-in
cookies, CSRF + security headers, Fastify MultipartModule.
Docs (Sep 2026): Techniques split into Application, Data,
HTTP; Observability and Reliability added.